

Donghang Zou's UChicago ADS Code Sample

This is an excerpt from a larger project: https://github.com/Booth-DH/Global_AI_Internship

County social conditions help identify where chronic disease and unmet needs coincide. The project uses five CDC PLACES releases from 2021 to 2025 and CDC SVI 2020 and 2022. It cleans and stacks releases into a county panel, correlates social and behavioral factors with diabetes and hypertension, groups counties with K-means, and maps descriptive profiles. This excerpt rebuilds the latest snapshot from raw CSVs. Food, housing, and transportation insecurity are the strongest diabetes correlates, with r of 0.937, 0.921, and 0.914.

The latest snapshot combines PLACES 2025 with SVI 2022.

GitHub: https://github.com/Booth-DH/Global_AI_Internship

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
ROOT = Path.cwd() if Path('raw_data').is_dir() else Path.cwd().parent
```

```
OUT = ROOT / 'code_sample'
```

```
outcomes = ['DIABETES', 'BPHIGH']
```

```
factors = ('LPA OBESITY CSMOKING ACCESS2 DEPRESSION SLEEP BINGE RPL_THEMES '
           'RPL_THEME1 RPL_THEME2 RPL_THEME3 RPL_THEME4 FOODINSECU '
           'HOUSINSECU LACKTRPT LONELINESS').split()
```

```
svi_cols = [f for f in factors if f.startswith('RPL')]
```

```
# 1. Load and type the data
```

```
# String identifiers preserve leading zeros in county FIPS codes.
```

```
places = pd.read_csv(ROOT / 'raw_data/places_county_2025.csv', low_memory=False,
```

```
dtype={'locationid': 'string', 'year': 'int16', 'data_value': 'float64'})
```

```
svi = pd.read_csv(ROOT / 'raw_data/svi_county_2022.csv', dtype={'FIPS': 'string'},
                  usecols=['FIPS', *svi_cols])
```

```
print(f'locationid: {places.locationid.dtype}; FIPS: {svi.FIPS.dtype}; '
      f'data_value: {places.data_value.dtype}')
```

```
locationid: string; FIPS: string; data_value: float64
```

Age-adjusted estimates form one row per county.

```
# 2. Prepare the county snapshot
```

```
def prepare_county_snapshot(places, svi):
```

```
    health = places.loc[(places.stateabbr != 'US') &
                       (places.data_value_type == 'Age-adjusted prevalence')].copy()
```

```
    health['FIPS'] = health.locationid.str.zfill(5)
```

```
    health = health.sort_values('year').drop_duplicates(['FIPS', 'measureid'], keep='last')
```

```
    wide = health.pivot(index='FIPS', columns='measureid', values='data_value')
```

```
    # SVI -999 is missing; a one-to-one join prevents duplicate county weights.
```

```
    svi = svi.assign(FIPS=svi.FIPS.str.zfill(5)).replace(-999, np.nan).set_index('FIPS')
```

```
    return wide.join(svi, how='inner', validate='one_to_one').dropna(
        subset=[*outcomes, 'RPL_THEMES'])
```

```
counties = prepare_county_snapshot(places, svi)
```

The function returns Pearson r and the county count for each factor.

```
# 3. Compare county associations
```

```
def compare_correlations(data, outcome, factors, sample_policy='pairwise'):
```

```
    assert sample_policy in {'pairwise', 'common'}
```

```
    values = data[[outcome, *factors]]
```

```
    # Common holds geography fixed; pairwise keeps all available pairs.
```

```
    if sample_policy == 'common':
```

```
        values = values.dropna()
```

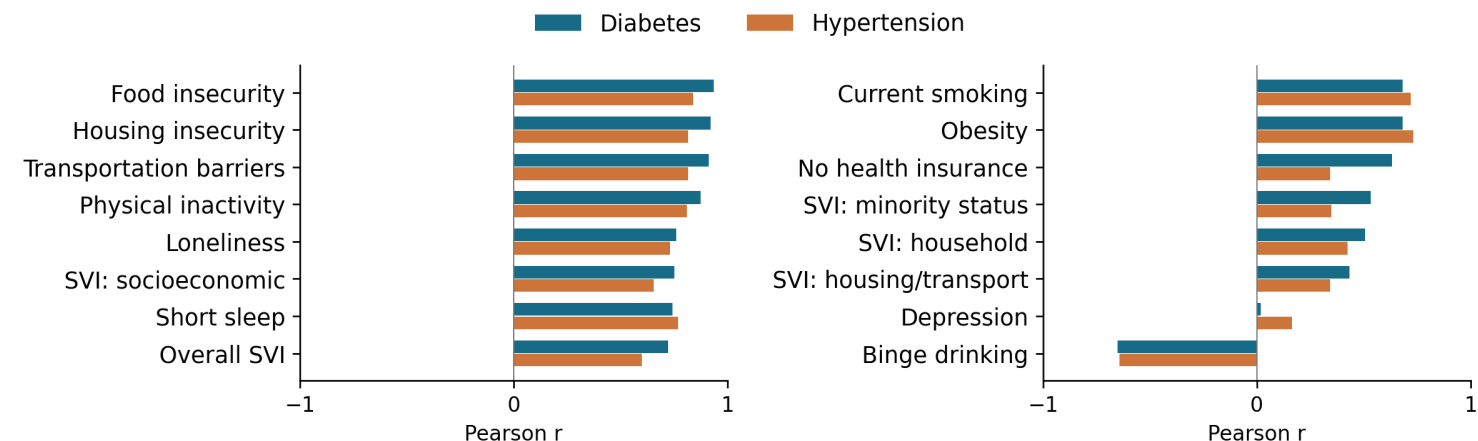
```
    rows = []
```

```
    for factor in factors:
```

```
        pairs = values[[outcome, factor]].dropna()
```

```
        rows.append((factor, pairs[factor].corr(pairs[outcome]), len(pairs)))
```

```
    return pd.DataFrame(rows, columns=['factor', 'r', 'n']).set_index('factor')
```



Donghang Zou's UChicago ADS Code Sample

The bars rank all 16 factors by diabetes r ; the table compares four factors, with common_r and common_n for diabetes.

```
# 4. Apply the comparison to both outcomes
results = {y: compare_correlations(counties, y, factors) for y in outcomes}
r = pd.DataFrame({y: result.r for y, result in results.items()})
shown = ['FOODINSECU', 'HOUSINSECU', 'LACKTRPT', 'LPA']
common = compare_correlations(counties, 'DIABETES', shown, sample_policy='common')
paired = r.loc[shown].join(results['DIABETES'].n)
common = common.rename(columns={'r': 'common_r', 'n': 'common_n'})
comparison = paired.join(common)
print(comparison.round(3).rename_axis(None).to_string())
plot_correlations(r)
```

Factors with mean absolute r of at least 0.40 join both outcomes; the table reports unweighted county means (% and SVI rank).

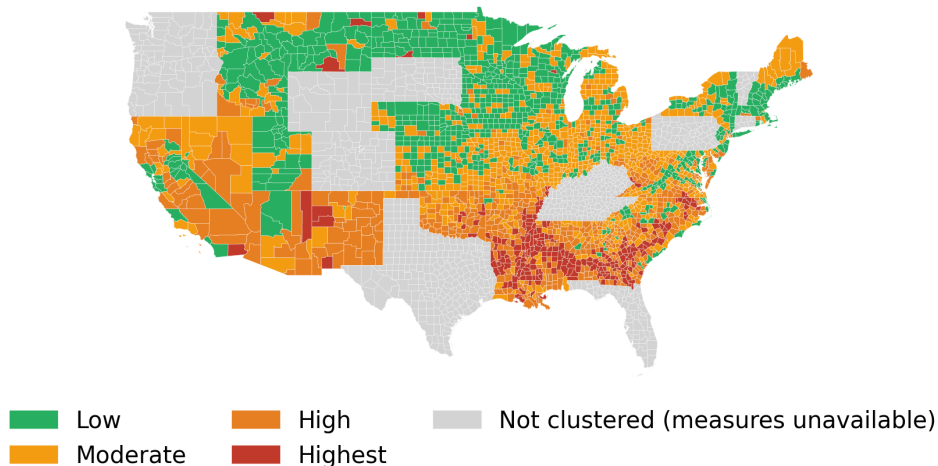
```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# 5. Group and map county profiles
strength = r.abs().mean(axis=1).sort_values(ascending=False)
features = outcomes + strength[strength >= 0.40].index.tolist()
complete = counties.dropna(subset=features).copy()
# Standardize before K-means so units do not determine distance.
z = StandardScaler().fit_transform(complete[features])
model = KMeans(n_clusters=4, random_state=42, n_init=10).fit(z)
anchors = [features.index(f) for f in [*outcomes, 'RPL_THEMES']]
order = np.argsort(model.cluster_centers_[:, anchors].mean(axis=1))
tiers = ['Low', 'Moderate', 'High', 'Highest']
labels = pd.Series(model.labels_, index=complete.index)
ordered_labels = labels.map(dict(zip(order, tiers)))
complete['tier'] = pd.Categorical(ordered_labels, categories=tiers, ordered=True)
summary = complete.groupby('tier', observed=True).agg(counties=('DIABETES', 'size'),
    diabetes=('DIABETES', 'mean'), hypertension=('BPHIGH', 'mean'), SVI=('RPL_THEMES', 'mean'))
print(f'{len(features)} features; {len(complete):,}/{len(counties):,} counties; '
    f'silhouette = {silhouette_score(z, model.labels_):.3f}')
print(summary.round(3).rename_axis(None).to_string())
plot_county_profiles(complete)
```

	DIABETES	BPHIGH	n	common_r	common_n
FOODINSECU	0.937	0.838	2299	0.937	2299
HOUSINSECU	0.921	0.816	2299	0.921	2299
LACKTRPT	0.914	0.816	2299	0.914	2299
LPA	0.873	0.810	2956	0.850	2299

16 features; 2,299/2,956 counties; silhouette = 0.203				
	counties	diabetes	hypertension	SVI
Low	765	9.090	29.758	0.190
Moderate	788	10.765	33.306	0.456
High	508	12.555	36.755	0.754
Highest	238	15.786	42.943	0.895

County profiles in the contiguous United States



Grey areas include 657 eligible counties in CO, FL, OR, SD, TN, TX, VT, WA, and WY lacking food, housing, transport, and loneliness measures; KY and PA have only five measures and no diabetes or hypertension estimates. Loving County, TX has suppressed outcomes. Nine CT planning regions lack cached geometry.

The three social-needs associations are stronger than physical inactivity. For inactivity, diabetes r falls from 0.873 on 2,956 counties to 0.850 on the 2,299-county common sample. Higher-burden profiles concentrate in the Deep South. These associations are not causal, measurement years differ (PLACES 2022/2023 and SVI 2022), and the profiles are descriptive and overlapping (silhouette 0.203).

Plotting helpers and the full runnable code are in the accompanying script and notebook.